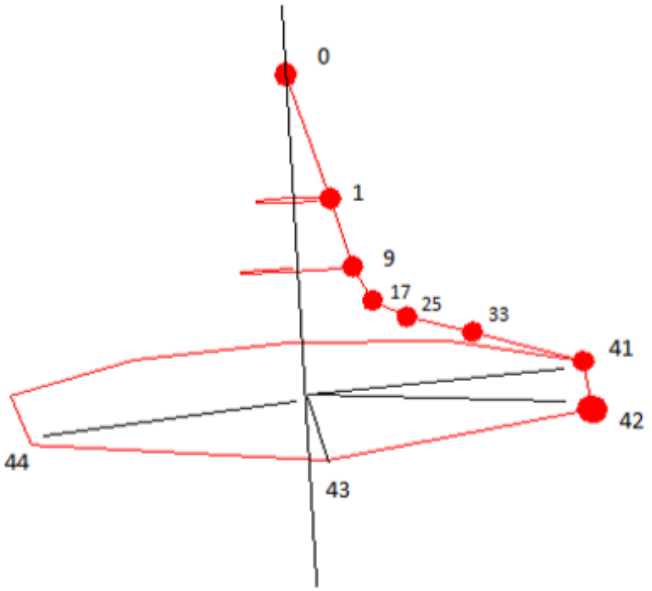
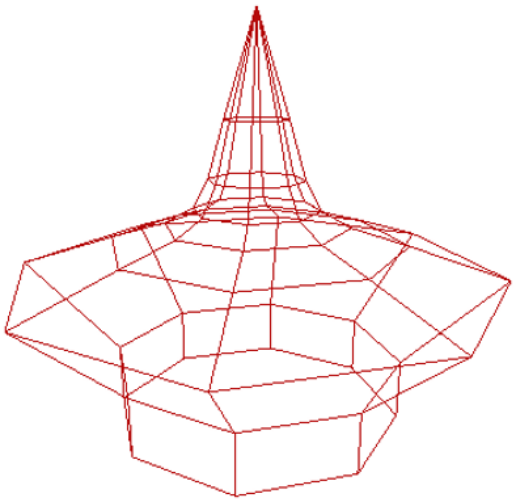
Sa se reprezinte un corp 3D prin rotirea unui fragment de curba libera (Bezier).



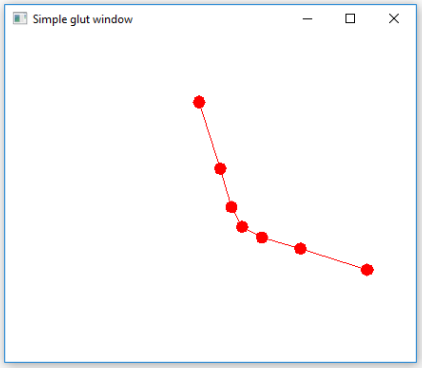
Intr-o prima etapa se determina coordonatele punctelor de pe curba Bezier (programul sursa de mai jos).

Vom numerota varfurile corpului cu 0,1,..,47, 48: primul varf (0) este varful din partea de sus; in continuare se calculeaza varfurile dispuse in 6 planuri paralele, fiecare plan fiind determinat de un octogon regulat (8 varfuri).

Primulplan de sus este format de nodurile 1,2,..8; planul urmator din nodurile 9,10,..16 iar ultimul plan din nodurile 41,42,…48. Astfel incat punctele de pe curba Bezier vor fi nodurile 0,1,9,17,25,33 si 41.



Partea de jos se completeaza prin scalarea ulimului octogon.




```
/* start 30.oct.2017 */

#include <GL/freeglut.h>
#include <stdlib.h>
#include <string.h>

#include <math.h>

struct point {
    double x;    double y;
};
struct POINT3D {
    double x;    double y;    double z;
};
struct POINT3D  cub[512];

// simple linear interpolation between two points
void lerp(point& dest, const point& a, const point& b, const float t)
{
    dest.x = a.x + (b.x-a.x)*t;
    dest.y = a.y + (b.y-a.y)*t;
}

// evaluate a point on a bezier-curve. t goes from 0 to 1.0
void bezier(point &dest, const point& a, const point& b, const point& c, const point& d, const float t)
{
    point ab,bc,cd,abbc,bccd;
    lerp(ab, a,b,t);
    lerp(bc, b,c,t);
    lerp(cd, c,d,t);
    lerp(abbc, ab,bc,t);
    lerp(bccd, bc,cd,t);
    lerp(dest, abbc,bccd,t);
}

//////////////////////////////////////////////
void genereaza() {
    double a=2*3.1415/8, sina, cosa;
    // punctele determinate de curba Bezier
    point A = { 0, 4 };
    point B = { 1.4, 0 };
    point C = { 0, 1.4 };
    point D = { 4, 0};

    int n=-1;  // nr total de puncte inregistrate
    // inregistrez coord pct Bezier
    for (int i=0; i<7; i++) // punctele de pe curba Bezier
    {
        point p;
        float t = static_cast<float>(i)/6.0;
        bezier(p, A, B, C, D, t);
        n++;
        cub[n].x = p.x;
        cub[n].y = p.y;
        cub[n].z = 0;

        // cout << "\n poz:"<<n<<"\t"<<cub[n].x<<","<<cub[n].y<<","<<cub[n].z;
        if(n!=0)
        {
            int k = n;            // pozitia nodului crt de pe curba Bezier
            // am inregistrat un punct de pe curba Bezier
            // mai sunt de inregistrat celelalte 7 puncte din planul orizontal
            // care vor constitui octogonul
            for(int j=1; j<8; j++)      // doar 7
            {
                n++;
                cub[n].y = cub[k].y;
                sina = sin(j*a);
                cosa = cos(j*a);
                cub[n].x = cub[k].x * cosa - cub[k].z * sina;
                cub[n].z = cub[k].x * sina + cub[k].z * cosa;
            }
        }
```

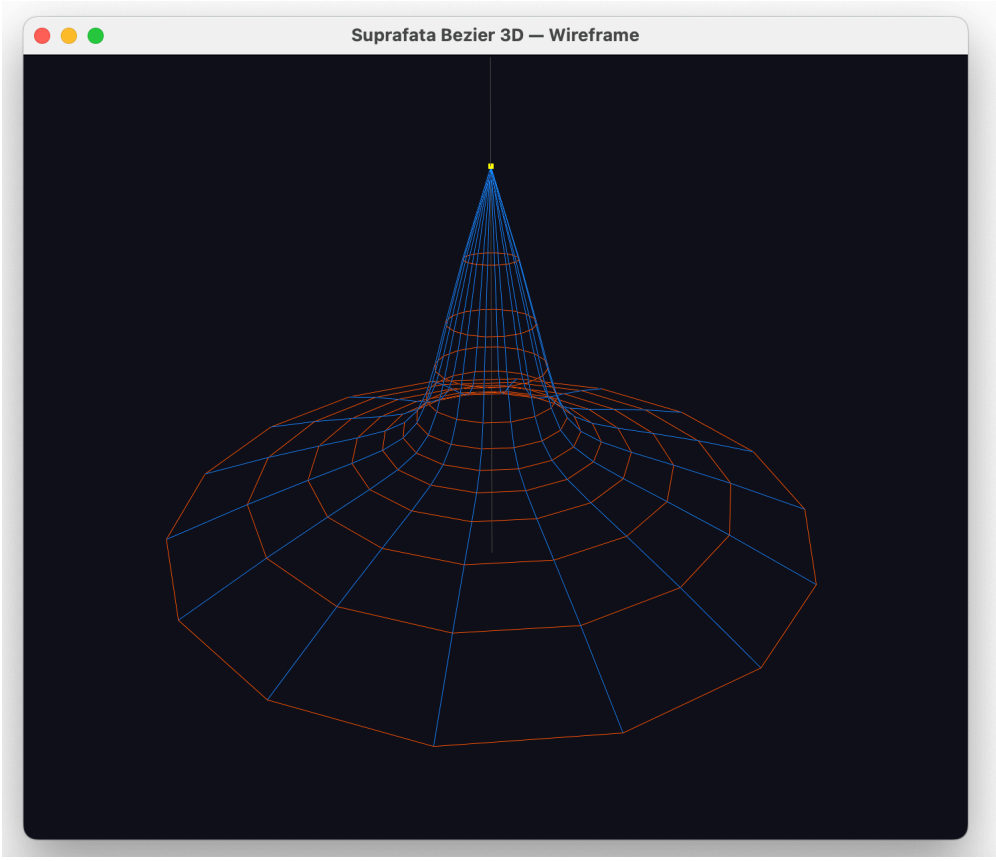
```
    }

}

/* Camera vectors */
struct Vector3d {
    float x, y, z;
} rot, eye;


/**
* Application initialisation. Setup keyboard input,
* mouse input, timer, camera and OpenGL.
*/
static void
init ()
{
    /* Init camera input */
    rot.x = 0.0f;
    eye.x = 0.0f;
    rot.y = 0.0f;
    eye.y = 0.0f;
    rot.z = 0.0f;
    eye.z = 10.0f;

    genereaza();

// Set background color
    glClearColor( 1.0, 1.0, 1.0, 1.0 );
}

/**
* GLUT's reshape callback function.
* Update the viewport and the projection matrix.
*/
static void
reshape (int w, int h)
{
    if (h == 0)
        h = 1;
    glViewport (0, 0, (GLsizei)w, (GLsizei)h);
    glMatrixMode (GL_PROJECTION);
    glLoadIdentity ();

    /*
    fovy - Specifies the field of view angle, in degrees, in the y direction.
    aspect - Specifies the aspect ratio that determines the field of view in the x direction.
The aspect ratio is the ratio of x (width) to y (height).
    zNear - Specifies the distance from the viewer to the near clipping plane (always positive).
    zFar - Specifies the distance from the viewer to the far clipping plane (always positive).
    */
    gluPerspective (60.0, (GLfloat)w/(GLfloat)h, 0.1, 50.0);
    glMatrixMode (GL_MODELVIEW);
    glLoadIdentity ();
    glutPostRedisplay ();
}


/**
* GLUT's display callback function.
* Render the main OpenGL scene.
*/
static void
display () {
    glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity ();

    /* Camera rotation */
    glTranslated (-eye.x, -eye.y, -eye.z);
    glRotated (rot.x, 1.0f, 0.0f, 0.0f);
    glRotated (rot.y, 0.0f, 1.0f, 0.0f);
    glRotated (rot.z, 0.0f, 0.0f, 1.0f);
```

```
    glBegin(GL_LINE_STRIP);
    /* traseaza conturul vertical k */
    glVertex3f(cub[0].x, cub[0].y, cub[0].z);
    for(int i=1; i<49; i+=8)
        glVertex3f(cub[i].x, cub[i].y, cub[i].z);
    glEnd();

    glColor3f(1., 0., 0.);
    glTranslatef(cub[0].x, cub[0].y, cub[0].z);
    glutSolidSphere(0.15,8,8);         // marcheaza primul nod
    glTranslatef(-cub[0].x, -cub[0].y, -cub[0].z);
    for(int i=1; i<=41; i+=8)
    {
        glTranslatef(cub[i].x, cub[i].y, cub[i].z);
        glutSolidSphere(0.15,8,8);     // marcheaza nodurile de pe curba Bezier
        glTranslatef(-cub[i].x, -cub[i].y, -cub[i].z);
    }
    glutSwapBuffers ();
}

int
main (int argc, char *argv[])
{
    glutInit (&argc, argv);
    glutInitWindowSize (640, 480);
    glutCreateWindow ("Simple glut window");
    init();
    glutReshapeFunc (reshape);
    glutDisplayFunc (display);
    glutMainLoop ();
    return 0;
}
```

```python
"""
Suprafata 3D generata prin revolutia unei curbe Bezier cubice
  - Wireframe interactiv cu OpenGL 1.x + GLFW
  - Mouse drag stanga  : rotatie
  - Mouse scroll       : zoom
  - Mouse drag dreapta : pan
  - R                  : reset camera
"""

import sys
import math
import glfw
from OpenGL.GL import *
from OpenGL.GLU import *

# ---------------------------------------------------------------------------
# Structuri de date
# ---------------------------------------------------------------------------
NUM_BEZIER_PTS  = 12   # puncte de-a lungul curbei Bezier
NUM_ROT_STEPS   = 8   # pasi de rotatie pentru revolutie
MAX_PTS         = NUM_BEZIER_PTS * NUM_ROT_STEPS + 1  # +1 pt varful de pe axa

cub = []   # lista de dict-uri {x, y, z}

# ---------------------------------------------------------------------------
# Bezier helpers
# ---------------------------------------------------------------------------
def lerp(a, b, t):
    return (a[0] + (b[0]-a[0])*t,
            a[1] + (b[1]-a[1])*t)

def bezier(a, b, c, d, t):
    ab   = lerp(a, b, t)
    bc   = lerp(b, c, t)
    cd   = lerp(c, d, t)
    abbc = lerp(ab, bc, t)
    bccd = lerp(bc, cd, t)
    return lerp(abbc, bccd, t)

# ---------------------------------------------------------------------------
# Generare suprafata prin revolutia curbei Bezier
# ---------------------------------------------------------------------------
def genereaza():
    global cub
    cub = []

    # Punctele de control Bezier (in planul XY, x>=0)
    A = (0.0,  4.0)   # varf (pe axa de simetrie)
    B = (1.4,  0.0)   # control
    C = (0.0,  1.4)   # control
    D = (4.0,  0.0)   # baza

    # Unghiul unui pas de rotatie (2*pi / NUM_ROT_STEPS)
    step_angle = 2.0 * math.pi / NUM_ROT_STEPS

    for i in range(NUM_BEZIER_PTS):
        t = i / float(NUM_BEZIER_PTS - 1)
        p = bezier(A, B, C, D, t)
        px, py = p[0], p[1]   # px = raza, py = inaltime

        if i == 0 and abs(px) < 1e-6:
            # Varful este pe axa: un singur punct
            cub.append({'x': 0.0, 'y': py, 'z': 0.0})
        else:
            # Generam NUM_ROT_STEPS puncte rotind in jurul axei Y
            for j in range(NUM_ROT_STEPS):
```

```
                angle = j * step_angle
                cub.append({
                    'x': px * math.cos(angle),
                    'y': py,
                    'z': px * math.sin(angle)
                })

    return cub

# ---------------------------------------------------------------------------
# Desenare wireframe
# ---------------------------------------------------------------------------
def draw_scene():
    # ---- Meridiane (contururi verticale de-a lungul curbei Bezier) ----
    glColor3f(0.15, 0.55, 1.0)   # albastru
    glLineWidth(1.4)

    for k in range(2):#NUM_ROT_STEPS
        glBegin(GL_LINE_STRIP)
        # Primul punct este varful (pe axa)
        glVertex3f(cub[0]['x'], cub[0]['y'], cub[0]['z'])
        start_ring = 0

        # Parcurgem inelele: fiecare inel are NUM_ROT_STEPS puncte
        for ring in range(start_ring, NUM_BEZIER_PTS):
            idx = start_ring + (ring - start_ring) * NUM_ROT_STEPS + k
            if idx < len(cub):
                p = cub[idx]
                glVertex3f(p['x'], p['y'], p['z'])
        glEnd()


    glColor3f(1.0, 1.0, 0.0)
    glPointSize(8)
    glBegin(GL_POINTS)
    glVertex3f(cub[0]['x'], cub[0]['y'], cub[0]['z'])
    glEnd()

    for k in range(2):
        start_ring = 0
        for ring in range(start_ring, NUM_BEZIER_PTS):
            idx = start_ring + (ring - start_ring) * NUM_ROT_STEPS + k
            if idx < len(cub):
                p = cub[idx]
                glColor3f(1.0, 1.0, 0.0)
                glPointSize(8)
                glBegin(GL_POINTS)
                glVertex3f(p['x'], p['y'], p['z'])
                glEnd()

    # ---- Paralele (contururi orizontale / inele) ----
    glColor3f(1.0, 0.35, 0.1)   # portocaliu
    glLineWidth(1.0)

    start_ring=1
    ring=11
    base = start_ring + (ring - start_ring) * NUM_ROT_STEPS
    glBegin(GL_LINE_LOOP)
    for k in range(NUM_ROT_STEPS):
        idx = base + k
        if idx < len(cub):
            p = cub[idx]
            glVertex3f(p['x'], p['y'], p['z'])
    glEnd()
```

```python
    # ---- Axa Y (referinta) ----
    glLineWidth(1.0)
    glBegin(GL_LINES)
    glColor3f(0.3, 0.3, 0.3)
    glVertex3f(0, -1, 0)
    glVertex3f(0,  5, 0)
    glEnd()

# ---------------------------------------------------------------------------
# Camera state
# ---------------------------------------------------------------------------
rot_x   =  25.0
rot_y   = -30.0
zoom    =  10.0
pan_x   =   0.0
pan_y   =   0.0

mouse_last_x = 0
mouse_last_y = 0
mouse_btn    = None   # 'left', 'right', None

# ---------------------------------------------------------------------------
# GLFW callbacks
# ---------------------------------------------------------------------------
def mouse_button_callback(window, button, action, mods):
    global mouse_btn, mouse_last_x, mouse_last_y
    x, y = glfw.get_cursor_pos(window)
    if action == glfw.PRESS:
        mouse_last_x, mouse_last_y = x, y
        if button == glfw.MOUSE_BUTTON_LEFT:
            mouse_btn = 'left'
        elif button == glfw.MOUSE_BUTTON_RIGHT:
            mouse_btn = 'right'
    elif action == glfw.RELEASE:
        mouse_btn = None

def cursor_pos_callback(window, x, y):
    global rot_x, rot_y, pan_x, pan_y, mouse_last_x, mouse_last_y
    dx = x - mouse_last_x
    dy = y - mouse_last_y
    mouse_last_x, mouse_last_y = x, y

    if mouse_btn == 'left':
        rot_y += dx * 0.5
        rot_x += dy * 0.5
    elif mouse_btn == 'right':
        pan_x += dx * 0.01
        pan_y -= dy * 0.01

def scroll_callback(window, xoff, yoff):
    global zoom
    zoom -= yoff * 0.5
    zoom = max(1.0, min(zoom, 50.0))

def key_callback(window, key, scancode, action, mods):
    global rot_x, rot_y, zoom, pan_x, pan_y
    if action == glfw.PRESS:
        if key == glfw.KEY_R:
            rot_x, rot_y = 25.0, -30.0
            zoom = 10.0
            pan_x = pan_y = 0.0
        elif key == glfw.KEY_ESCAPE or key == glfw.KEY_Q:
            glfw.set_window_should_close(window, True)
```

```python
# ---------------------------------------------------------------------------
# Render
# ---------------------------------------------------------------------------
def display(window):
    w, h = glfw.get_framebuffer_size(window)
    if h == 0:
        h = 1
    glViewport(0, 0, w, h)

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    # Proiectie perspectiva
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluPerspective(60.0, w / h, 0.1, 200.0)

    # Camera
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    glTranslatef(pan_x, pan_y, -zoom)
    glRotatef(rot_x, 1, 0, 0)
    glRotatef(rot_y, 0, 1, 0)
    # Centram obiectul in Y
    glTranslatef(0, -2.0, 0)

    draw_scene()

    glfw.swap_buffers(window)

# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    if not glfw.init():
        print("Nu s-a putut initializa GLFW")
        sys.exit(1)

    # OpenGL 1.x (compatibility)
    glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 1)
    glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 4)

    window = glfw.create_window(900, 700, "Suprafata Bezier 3D — Wireframe", None, None)
    if not window:
        glfw.terminate()
        print("Nu s-a putut crea fereastra GLFW")
        sys.exit(1)

    glfw.make_context_current(window)
    glfw.set_mouse_button_callback(window, mouse_button_callback)
    glfw.set_cursor_pos_callback(window, cursor_pos_callback)
    glfw.set_scroll_callback(window, scroll_callback)
    glfw.set_key_callback(window, key_callback)

    # OpenGL setup
    glClearColor(0.06, 0.06, 0.10, 1.0)   # fundal inchis
    glEnable(GL_DEPTH_TEST)
    glEnable(GL_LINE_SMOOTH)
    glHint(GL_LINE_SMOOTH_HINT, GL_NICEST)
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
    glEnable(GL_BLEND)

    genereaza()

    print("=" * 52)
    print("  Suprafata 3D — Wireframe")
    print("=" * 52)
```

```python
    print("  Mouse stanga drag  : rotatie")
    print("  Mouse dreapta drag : pan")
    print("  Scroll             : zoom")
    print("  R                  : reset camera")
    print("  ESC / Q            : iesire")
    print("=" * 52)

    while not glfw.window_should_close(window):
        glfw.poll_events()
        display(window)

    glfw.terminate()

if __name__ == '__main__':
    main()
```